COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE-010	**SECTION:**	CPE21S4
DATE PERFORMED:	8/6/2026	**DATE SUBMITTED:**	8/8/2026
NAME:	Almirol Joel A	**INSTRUCTOR:**	Engr. Jim lord Quejao

ACTIVITY NO.5 **Queues**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILOA

```cpp
#include <iostream>
#include <queue>

void display(std::queue<int> q)
{
    std::queue<int> c = q;

    while (!c.empty())
    {
        std::cout << " " << c.front();
        c.pop();
    }

    std::cout << "\n";
}

int main()
{
    std::queue<int> a;

    // Insert elements into the queue
    a.push(10);
    a.push(20);
    a.push(30);

    std::cout << "The queue a is :";
    display(a);
```

```cpp
    std::cout << "The queue a is :";
    display(a);

    std::cout << "a.empty() : " << a.empty() << "\n";
    std::cout << "a.size()  : " << a.size() << "\n";
    std::cout << "a.front() : " << a.front() << "\n";
    std::cout << "a.back()  : " << a.back() << "\n";

    std::cout << "After a.pop() : ";
    a.pop();
    display(a);

    a.push(40);

    std::cout << "The queue a is : ";
    display(a);

    return 0;
}
```

Analysis:

the features of the queue one after another. In the main method, it declares a queue known as a and enqueues three values namely 10, 20, and 30 into the queue. To print the queue elements without erasing the initial values from the queue, it uses a special method known as display() which is responsible for creating a copy of the queue (c) and emptying this queue only. Then, it evaluates the conditions of the queue through some special methods such as empty() for checking the presence of the elements in the queue, size() for counting the elements, and front() and back() for retrieving the first and the last elements respectively. Lastly, it demonstrates the process of dequeuing and enqueueing the elements using the methods pop(), push(), displaying the resulting queue (20 30 40).

```
The queue a is : 10 20 30
a.empty() : 0
a.size()  : 3
a.front() : 10
a.back()  : 30
After a.pop() :  20 30
The queue a is :  20 30 40
```

ILOB

Main.cpp

```cpp
#include <iostream>
#include "queueLL.h"
#include "queueArr.h"

int main() {

    qNode<int>* front = nullptr;
    qNode<int>* back = nullptr;

    enqueue(1, &front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << "\n";
    enqueue(2, &front, &back);
    enqueue(3, &front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << "\n";

    display(front);
    dequeue(&front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << "\n";
    dequeue(&front, &back);
    std::cout << "front: " << front->data << " back: " << back->data << "\n";
    dequeue(&front, &back);
    dequeue(&front, &back);
    display(front);

    display(front);
```

```cpp
    //object cration
    queueArr<int> q(6);

    q.Enqueue(1);
    std::cout<<"Front: " <<q.Front() << " Back: "<< q.Back() << " Size: " << q.Size()<<std::endl;
    q.Enqueue(2);
    std::cout<<"Front: " <<q.Front() << " Back: "<< q.Back() << " Size: " << q.Size() << std::endl;
    q.Enqueue(3);
    std::cout<<"Front: " <<q.Front() << " Back: "<< q.Back() << " Size: " << q.Size() <<std::endl;
    q.Enqueue(4);
    std::cout<<"Front: " <<q.Front() << " Back: "<< q.Back() << " Size: " << q.Size() <<std::endl;
    q.Enqueue(5);
    std::cout<<"Front: " <<q.Front() << " Back: "<< q.Back() << " Size: " << q.Size() <<std::endl;
    q.Enqueue(6);
    std::cout<<"Front: " <<q.Front() << " Back: "<< q.Back() << " Size: " << q.Size() <<std::endl;
    q.Enqueue(7);
    std::cout<<"Front: " <<q.Front() << " Back: "<< q.Back() << " Size: " << q.Size() <<std::endl;
    q.Dequeue();
    std::cout<<"Front: " <<q.Front() << " Back: "<< q.Back() << " Size: " << q.Size() <<std::endl;
    q.Dequeue();
    q.Dequeue();
    q.Dequeue();
    q.Dequeue();
    q.Dequeue();
    q.Dequeue();
    q.Dequeue();
    q.Dequeue();
```

Analysis:

We have two custom implementations of a queue—one implemented using a linked list (queueLL.h) and the other using an array (queueArr.h). The first task involves raw pointers (front and back) and shows us how to perform enqueue, dequeue, and display operations using a linked list queue and keeping track of the node pointers ourselves during every insertion and deletion operation. In the second part, an array-based queue class is instantiated (queueWithArr q(6)) and elements (1 to 8) are pushed to perform queue operations.

```
    return 0;
}
```

queueLL.h

```
#ifndef QUEUELL_H
#define QUEUELL_H

#include <iostream>

template <typename T>
class qNode {
public:
    T data;
    qNode *next;
};

// creating the new node
template <typename T>
qNode<T> *new_node(T newData) {
    qNode<T> *newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = NULL;
    return newNode;
}

template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr) {
    // create a new node:
    qNode<T>* newNode = new_node(newData);

    // inserting into an empty queue
    // how do we know that the queue is empty?
```

```
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr) {
    if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
        // assign the frontPtr to the newNode
        (*frontPtr) = newNode;
        // assign the backPtr to the newNode
        (*backPtr) = newNode;
        return;
    }

    // point the back next to the newNode:
    (*backPtr)->next = newNode;
    // the newnode will be the new backPtr
    (*backPtr) = newNode;
}

template <typename T>
void dequeue(qNode<T> **frontPtr, qNode<T> **backPtr) {
    // how to determine if the queue is empty?
    if ((*frontPtr) == nullptr && (*backPtr) == nullptr) {
        std::cout << "The queue is empty. Nothing to dequeue\n.";
        return;
    }

    // creating a temporary pointer:
    qNode<T>* tempPtr = nullptr;
    // store the frontPtr to the tempPtr;
    tempPtr = (*frontPtr);
```

Analysis:

the generic linked list queue by using templated functions with double pointers (qNode<T>**). The code starts with a generic qNode<T> struct and a function called new_node() for allocating memory to the nodes, after which come three functions to perform the basic operations on the queue: enqueue(), dequeue(), and display(). Enqueue() function is used to dynamically add new nodes to the end of the queue, updating backPtr (or both the pointers if the list was empty previously), and dequeue() function is used to delete nodes from the start of the queue, ensuring proper memory allocation is done.

```
    // check if queue has only 1 node left???
    if ((*frontPtr) == (*backPtr)) {
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete tempPtr;
        return;
    }

    // update the frontPtr
    (*frontPtr) = (*frontPtr)->next;
    // point the tempPtr ->next to null
    tempPtr->next = nullptr;
    delete tempPtr;
}

// display all items in the queue: check the traverse in singlyLL
template <typename T>
void display(qNode<T>* frontPtr) {
    // catch if the queue is empty
    if (frontPtr == nullptr) {
        std::cout << "The queue is empty.\n";
        return;
    }

    // temp pointer to store the front Ptr
    qNode<T>* current = frontPtr;
```

```
    // temp pointer to store the front Ptr
    qNode<T>* current = frontPtr;

    // create a loop that will print the tempPtr->data
    while (current != nullptr) {
        std::cout << current->data << " ";
        current = current->next;
    }
    std::cout << std::endl;
}

#endif
```

```
front: 1 back: 1
front: 1 back: 3
1 2 3
front: 2 back: 3
front: 3 back: 3
The queue is empty. Nothing to dequeue
.The queue is empty.
```

queueArr.h

```h
#ifndef QUEUEARR_H
#define QUEUEARR_H

#include <iostream>

template <typename T>
class queueArr {
private:
    // pointer to dynamically allocated array
    T* q_array;
    size_t q_capacity;
    size_t q_size;
    int q_front;
    int q_back;

public:
    // constructor
    queueArr(size_t capacity = 10);

    bool Empty();
    size_t Size();
    void Clear();

    T Front();
    T Back();

    void Enqueue(T value);
    void Dequeue();
```

```h
    // copy constructor:
    queueArr(const queueArr& other);

    // copy assignment operator
    queueArr& operator=(const queueArr& other);

    // destructor
    ~queueArr();
};

// constructor
template <typename T>
queueArr<T>::queueArr(size_t capacity) {
    q_capacity = capacity;
    q_size = 0;
    q_front = 0;
    q_back = -1;
    q_array = new T[q_capacity];
}

template <typename T>
bool queueArr<T>::Empty() {
    return q_size == 0;
}

template <typename T>
size_t queueArr<T>::Size() {
```

Analysis:

The queueArr<T>, a fixed-size circular queue built on top of a dynamically created array. QueueArr handles indexing through the q_front and q_back member variables, using the modulus operator (%q_capacity) when performing Enqueue() and Dequeue() operations to support circular logic without needing to shift memory locations. In addition to the Empty(), Size(), and Clear() helper functions to check and clear the state of the queue, there are accessor functions Front() and Back() which return the boundary elements while ensuring that no underflow occurs. There is also a destructor (~queueArr()) which frees the dynamic array (delete [] q_array).

```
    return q_size;
}

template <typename T>
void queueArr<T>::Clear() {
    q_size = 0;
    q_front = 0;
    q_back = -1;
}

template <typename T>
T queueArr<T>::Front() {
    // what will happen if the queue is already empty?
    if (Empty()) {
        std::cout << "The Queue is Empty.\n";
        return T();
    }

    return q_array[q_front];
}

// back
template <typename T>
T queueArr<T>::Back() {
    if (Empty()) {
        std::cout << "The Queue is Empty.\n";
        return T();
```

```
T queueArr<T>::Back() {
    }

    return q_array[q_back];
}

template <typename T>
void queueArr<T>::Enqueue(T value) {
    // check if the queue is full
    if (q_size == q_capacity) {
        std::cout << "Queue is full\n";
        return;
    }

    // compute for the q_back for circular (use the modulo operator)
    q_back = (q_back + 1) % q_capacity;
    // assign the q_array with the updated q_back to the value
    q_array[q_back] = value;
    // increment the q_size
    q_size++;
    return;
}

template <typename T>
void queueArr<T>::Dequeue() {
    // check if it is empty
    if (Empty()) {
        std::cout << "the queue is empty\n ";
```

```
        return;
    }

    // move q_front circularly
    q_front = (q_front + 1) % q_capacity;

    // decrement the q_size
    q_size--;

    // reset index when queue becomes empty
    if (Empty()) {
        Clear();
        return;
    }
}

template <typename T>
queueArr<T>::~queueArr() {
    delete[] q_array;
}

#endif // QUEUEARR_H
```

```
The queue is empty.
Front: 1 Back: 1 Size: 1
Front: 1 Back: 2 Size: 2
Front: 1 Back: 3 Size: 3
Front: 1 Back: 4 Size: 4
Front: 1 Back: 5 Size: 5
Front: 1 Back: 6 Size: 6
Queue is full
Front: 1 Back: 6 Size: 6
Front: 2 Back: 6 Size: 5
the queue is empty
 the queue is empty
 the queue is empty
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

4. SUPPLEMENTARY ACTIVITY

ILO C: Solve problems using queue implementation

Problem Title: Shared Printer Simulation using Queues
Problem Definition: In this activity, we'll simulate a queue for a shared printer in an office. In any corporate office, usually, the printer is shared across the whole floor in the printer room. All the computers in this room are connected to the same printer. But a printer can do only one printing job at any point in time, and it also takes some time to complete any job. In the meantime, some other user can send another print request. In such a case, a printer needs to store all the pending jobs somewhere so that it can take them up once its current task is done.

Perform the following steps to solve the activity. Make sure that you include a screenshot of the source code for each. From the get-go: You must choose whether you are making a linked list or array implementation.
You may NOT use the STL.

1. Create a class called Job (comprising an ID for the job, the name of the user who submitted it, and the number of pages).
2. Create a class called Printer. This will provide an interface to add new jobs and process all the jobs added so far.
3. To implement the printer class, it will need to store all the pending jobs. We'll implement a very basic strategy – first come, first served. Whoever submits the job first will be the first to get the job done.
4. Finally, simulate a scenario where multiple people are adding jobs to the printer, and the printer is processing them one by one.
5. Defend your choice of internal representation: Why did you use arrays or linked lists?

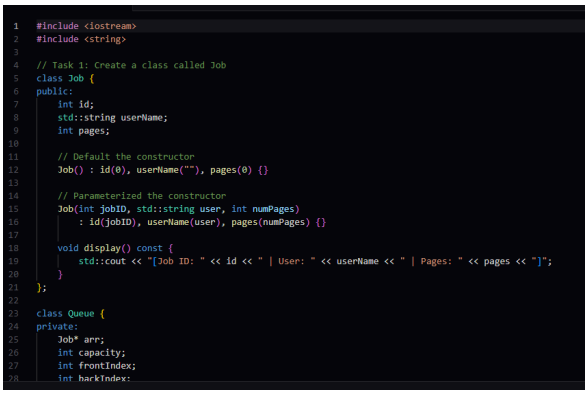
Output Analysis:

- Provide the output after performing each task above.
- Include your analysis: focus on why you think the output is the way it is.

5. Conclusions/Learnings:

Provide the following:

- Summary of lessons learned
- Analysis of the procedure
- Analysis of the supplementary activity
- Concluding statement / Feedback: How well did you think you did in this activity? What are your areas for improvement?



```cpp
#include <iostream>
#include <string>

// Task 1: Create a class called Job
class Job {
public:
    int id;
    std::string userName;
    int pages;

    // Default the constructor
    Job() : id(0), userName(""), pages(0) {}

    // Parameterized the constructor
    Job(int jobID, std::string user, int numPages)
        : id(jobID), userName(user), pages(numPages) {}

    void display() const {
        std::cout << "[Job ID: " << id << " | User: " << userName << " | Pages: " << pages << "]";
    }
};

class Queue {
private:
    Job* arr;
    int capacity;
    int frontIndex;
    int backIndex;
```

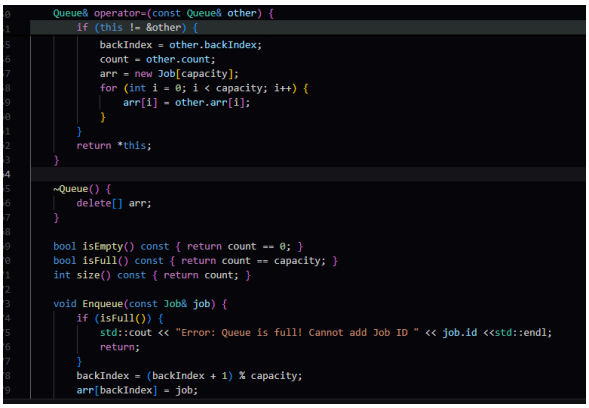


```cpp
    int backIndex;
    int count;

public:
    // The constructor
    Queue(int cap = 10) : capacity(cap), frontIndex(0), backIndex(-1), count(0) {
        arr = new Job[capacity];
    }

    // Copy the Constructor
    Queue(const Queue& other) {
        capacity = other.capacity;
        frontIndex = other.frontIndex;
        backIndex = other.backIndex;
        count = other.count;
        arr = new Job[capacity];
        for (int i = 0; i < capacity; i++) {
            arr[i] = other.arr[i];
        }
    }

    // Copy the Assignment Operator
    Queue& operator=(const Queue& other) {
        if (this != &other) {
            delete[] arr;
            capacity = other.capacity;
            frontIndex = other.frontIndex;
```



```cpp
    Queue& operator=(const Queue& other) {
        if (this != &other) {
            backIndex = other.backIndex;
            count = other.count;
            arr = new Job[capacity];
            for (int i = 0; i < capacity; i++) {
                arr[i] = other.arr[i];
            }
        }
        return *this;
    }

    ~Queue() {
        delete[] arr;
    }

    bool isEmpty() const { return count == 0; }
    bool isFull() const { return count == capacity; }
    int size() const { return count; }

    void Enqueue(const Job& job) {
        if (isFull()) {
            std::cout << "Error: Queue is full! Cannot add Job ID " << job.id <<std::endl;
            return;
        }
        backIndex = (backIndex + 1) % capacity;
        arr[backIndex] = job;
```

Analysis:

A printing job queue program based on a custom circular queue implemented using a custom array to handle printing requests in FIFO order. The program creates a structure called a Job that stores task information like task id, task user name, and number of pages along with a custom Queue that handles memory allocation dynamically, object copy through the rule of three, and indexing using modulo. Using the structures mentioned above, the Printer class wraps up the queue for handling high-level tasks where users can use the addJob() method to enqueue their tasks and processJobs() to print and dequeue each task.

```
    }

    void Dequeue() {
        if (isEmpty()) {
            std::cout << "Error: Queue is empty!\n";
            return;
        }
        frontIndex = (frontIndex + 1) % capacity;
        count--;
    }

    Job Front() const {
        if (!isEmpty()) return arr[frontIndex];
        return Job();
    }

    Job Back() const {
        if (!isEmpty()) return arr[backIndex];
        return Job();
    }
};

// Task 2 and 3 The create a class called Printer using the queue
class Printer {
private:
    Queue jobQueue;
```

```
public:
    Printer(int capacity = 10) : jobQueue(capacity) {}

    void addJob(const Job& job) {
        std::cout << "Adding to printer: ";
        job.display();
        std::cout << "\n";
        jobQueue.Enqueue(job);
    }

    // Task 4:
    void processJobs() {
        std::cout << "\n--- Starting Printer Processing ---\n";
        while (!jobQueue.isEmpty()) {
            Job currentJob = jobQueue.Front();
            std::cout << "Printing -> ";
            currentJob.display();
            std::cout << " ... [Completed]\n";
            jobQueue.Dequeue();
        }
        std::cout << "All jobs processed. Printer queue is clear.\n";
    }
};

int main() {
    Printer officePrinter(5);
```

```
    std::cout << "--- SIMULATING MULTIPLE USERS ADDING PRINT JOBS ---\n";
    officePrinter.addJob(Job(101, "alyanah", 5));
    officePrinter.addJob(Job(102, "katie", 12));
    officePrinter.addJob(Job(103, "Prored", 3));
    officePrinter.addJob(Job(104, "Akane", 20));

    officePrinter.processJobs();

    return 0;
}
```

```
--- SIMULATING MULTIPLE USERS ADDING PRINT JOBS ---
Adding to printer: [Job ID: 101 | User: alyanah | Pages: 5]
Adding to printer: [Job ID: 102 | User: katie | Pages: 12]
Adding to printer: [Job ID: 103 | User: Prored | Pages: 3]
Adding to printer: [Job ID: 104 | User: Akane | Pages: 20]

--- Starting Printer Processing ---
Printing -> [Job ID: 101 | User: alyanah | Pages: 5] ...
[Completed]
Printing -> [Job ID: 102 | User: katie | Pages: 12] ...
[Completed]
Printing -> [Job ID: 103 | User: Prored | Pages: 3] ...
[Completed]
Printing -> [Job ID: 104 | User: Akane | Pages: 20] ...
[Completed]
All jobs processed. Printer queue is clear.
```

Conclusion

This conclusion has helped me gain an understanding of how the First-In, First-Out (FIFO) Queue data structure works and the various ways it can be implemented in C++. This lab helped me practice three different kinds of implementations of this data structure: through the use of the std::queue library, through the design of a pointer-based dynamic linked list queue (queueLL.h), and lastly by building a fixed capacity circular array queue (queueArr.h). Apart from practicing basic functions such as enqueue, dequeue, and examining the boundaries of the queue, I was able to develop C++ engineering skills related to dynamic memory allocation, double-pointer management (qNode<T>**), circular array index wraparound using modulo operation, and resource management using Rule of Three.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or

completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.